

CPSC 250

Airline Seat Reservation System

Edward Brash

1 Program Goal

In this example, we design a simple airline seat reservation system.

The goal is to use classes and objects to model seats on an airplane.

Suppose the airplane has five seats. We will create a list containing five `Seat` objects.

2 Step 1: Identify Internal Variables

For each seat reservation, we want to store:

1. first name,
2. last name,
3. amount paid.

These become the internal variables of the class.

3 Step 2: Identify Class Methods

The `Seat` class should provide methods that allow us to:

1. reserve a seat,
2. unreserve a seat,
3. check whether the seat is empty,
4. print seat information.

The user should not directly manipulate the internal variables.

Instead, the user should interact with the object through methods.

4 The Seat Class

A reasonable `Seat` class is:

```

class Seat:

    def __init__(self):
        self.first_name = ""
        self.last_name = ""
        self.amount_paid = 0.0

    def reserve(self, first_name, last_name, amount_paid):
        self.first_name = first_name
        self.last_name = last_name
        self.amount_paid = amount_paid

    def unreserve(self):
        self.first_name = ""
        self.last_name = ""
        self.amount_paid = 0.0

    def is_empty(self):
        return self.first_name == "" and self.last_name == ""

    def print_seat(self):
        print(self.first_name, self.last_name, self.amount_paid)

```

5 Creating Seat Objects

Suppose:

```

num_seats = 5
available_seats = []

```

We can create five seat objects using a loop:

```

for i in range(num_seats):
    available_seats.append(Seat())

```

Now `available_seats` is a list of objects:

```

[Seat(), Seat(), Seat(), Seat(), Seat()]

```

Initially, all seats are empty.

6 Lists of Objects

This is an important idea:

A list can contain objects, not just primitive values.

Each element of the list is a separate `Seat` object.

For example:

```

available_seats[0]

```

is the first seat object.

```

available_seats[0].is_empty()

```

calls the `is_empty()` method for the first seat.

7 Printing Seat Status

We can write a function to print the current status of all seats.

```
def print_seats(seats):  
    seat_status = []  
  
    for i in range(len(seats)):  
        seat_status.append(seats[i].is_empty())  
  
    print("Current seat status")  
    print(seat_status)
```

Initially this might print:

```
[True, True, True, True, True]
```

because all seats are empty.

8 Printing the Passenger List

We can also print only the seats that are occupied.

```
def print_passenger_list(seats):  
    for i in range(len(seats)):  
        if not seats[i].is_empty():  
            print(f"{i}: ", end="")  
            seats[i].print_seat()
```

This demonstrates method calls on objects stored in a list.

9 Menu Driver

A menu driver repeatedly asks the user what task to perform.

The options are:

Command	Meaning
p	print the seat status
l	print the passenger list
r	reserve a new seat
q	quit

10 Menu Loop

```
command = input("Enter command: ")  
  
while command != "q":
```

```

if command == "p":
    print_seats(available_seats)

elif command == "l":
    print_passenger_list(available_seats)

elif command == "r":
    reserve_seat(available_seats)

else:
    print("Invalid command")

command = input("Enter command: ")

```

11 Reserving a Seat

The `reserve_seat()` function asks for a seat number and passenger information.

```

def reserve_seat(seats):

    seat_num = int(input("Enter seat number: "))

    if not seats[seat_num].is_empty():

        print("Seat not empty")

    else:

        first_name = input("Enter first name: ")
        last_name = input("Enter last name: ")
        paid = float(input("Enter amount paid: "))

        seats[seat_num].reserve(first_name, last_name, paid)

```

12 Complete Main Program

```

num_seats = 5
available_seats = []

for i in range(num_seats):
    available_seats.append(Seat())

command = input("Enter command: ")

while command != "q":

    if command == "p":
        print_seats(available_seats)

    elif command == "l":
        print_passenger_list(available_seats)

```

```
elif command == "r":
    reserve_seat(available_seats)

else:
    print("Invalid command")

command = input("Enter command: ")
```

13 Key Takeaways

- Classes are useful for modeling real-world objects.
- A seat object stores passenger information.
- Lists can store objects.
- Functions can receive lists of objects as arguments.
- Menu-driven programs use loops and branching.
- Encapsulation keeps object data organized and controlled.